DES3UE WS 2026 Übung 6

Abgabetermin: siehe e-Learning, Abgabeform elektronisch

☐ **DES3UEG1 (Niklas)** **Name** _____ **Aufwand in h** _____
☐ **DES3UEG2 (Niklas)** **Punkte** _____ **Kurzzeichen Tutor*in** _____
☐ **DES3UEG3 (Teufl)**

Ziel dieser Übung ist die Einführung in die Grundlagen von PL/SQL und die Erstellung von gespeicherten Prozeduren in der Datenbank.

Zusätzliche Hinweise

Aufgaben 1 und 2 bauen aufeinander auf. Fügen Sie für jedes Beispiel (auch Unterpunkte) den entsprechenden PL/SQL-Code in Ihr Abgabedokument ein. Geben Sie alle **Zwischenergebnisse** ab und kennzeichnen Sie die Ausarbeitung der jeweiligen Aufgabe!

1. PL/SQL Grundlagen **(7,5 Punkte** – 2 + 3 + 2 + 0,5**)**

Die Geschäftsführung von Sakila überlegt, Strafgebühren (Penalty) für verspätete Rückgaben einzuführen. Als Entscheidungsgrundlage soll zunächst für alle Filme die Anzahl der verspäteten Rückgaben ermittelt werden. Jeder Film hat eine erlaubte Leihdauer in Tagen, die in der Spalte `rental_duration` der Tabelle `film` gespeichert ist. Ein Verleihvorgang aus der Tabelle `rental` gilt als verspätet, wenn die Anzahl der angebrochenen Leihtage, also `CEIL(return_date - rental_date)`, größer als die erlaubte Leihdauer des Films ist.

1. Erstellen Sie zunächst eine SQL-Abfrage, die für **alle** Filme folgende Daten ausgibt:
   - ID,
   - Titel,
   - erlaubte Leihdauer,
   - die Anzahl der Verleihvorgänge,
   - die Anzal der davon zurückgegebenen Verleihvorgänge,
   - die Anzahl der verspäteten Rückgaben.

   Auch Filme, von denen es kein Exemplar gibt oder die noch nie verliehen wurden, sollen aufscheinen (mit 0 Verleihvorgängen).

2. Die Geschäftsführung möchte diese Auswertung zusätzlich für **einzelne Filme** abrufen und dabei eine Empfehlung erhalten. Schreiben Sie dazu einen anonymen PL/SQL-Block, der für einen Film folgende Daten ausgibt:

   - Titel,
   - erlaubte Leihdauer,
   - die Anzahl der Verleihvorgänge und der davon zurückgegebenen Vorgänge,
   - die Anzahl der verspäteten Rückgaben,
   - prozentualer Anteil der verspäteten Rückgaben in Bezug auf zurückgegebene Vorgänge (gerundet auf eine Nachkommastelle) sowie
   - eine Empfehlung: ab einem Anteil über 30 Prozent lautet sie „Penalty dringend empfohlen“, bei einem Anteil über 10 Prozent „Penalty prüfen“ und andernfalls „keine Maßnahme nötig“ (kann mit einem `CASE`-Ausdruck ermittelt werden). case

   Verwenden Sie dazu Ihre Abfrage aus 1.1, eingeschränkt auf einen Film, und lesen Sie alle benötigten Werte mit einem `SELECT ... INTO` in Variablen ein. Die Nummer des Films soll über `&film_id` eingegeben, die ermittelten Daten über `DBMS_OUTPUT` ausgegeben werden.

Beispielausgabe:

```
Film: ACADEMY DINOSAUR (erlaubte Leihdauer: 6 Tage)
Verleihvorgänge: 23, davon zurückgegeben: 23
Verspätet zurückgegeben: 3 (13 %)
Empfehlung: Penalty prüfen
```

3. Testen Sie Ihren anonymen Block aus 1.2 mit verschieden Filmen, um zumindest alle drei Empfehlungsvarianten („Penalty dringend empfohlen“, „Penalty prüfen“ und „keine Maßnahme nötig“) zeigen zu können. Testen Sie zudem folgende Randfälle: Einen Film, von dem es kein einziges Exemplar gibt (z.B. ID 108), sowie einen nicht existierenden Film (z.B. ID 1001). Kommentieren Sie Ihre Tests und erklären Sie, warum Film 1001 zu einem Fehler führt, ein Film ohne Exemplar beim Abfragen jedoch nicht, und welches Problem beim Film ohne Exemplar stattdessen auftreten kann.

4. Bevor die Penalty eingeführt wird, soll das neue Verleihsystem in einem **Pilotbetrieb** evaluiert werden, der parallel zum Echtbetrieb läuft. Legen Sie dafür eine Kopie aller relevanten Daten der Tabelle `rental` mit dem Namen `evaluation_rental` an. Damit Verleihvorgänge aus dem Pilotbetrieb von den historischen unterschieden werden können, bekommen diese IDs ab 100000 vergeben. Zusätzlich wird die Tabelle `evaluation_rental` um die Spalte `penalty` zur Speicherung einer etwaigen Strafgebühr sowie um zwei Check Constraints erweitert.
Sehen Sie das Skript `UE06_01_04.sql` durch und **führen** Sie **es aus**. Das Skript erfüllt bereits alle oben genannten Anforderungen.

```sql
-- DROP TABLE evaluation_rental;
-- DROP SEQUENCE evaluation_rental_seq;

-- Erstellen der Tabelle evaluation_rental (als Kopie von rental)
CREATE TABLE evaluation_rental AS
SELECT rental_id, rental_date, inventory_id, customer_id, return_date,
staff_id
FROM rental;
ALTER TABLE evaluation_rental ADD PRIMARY KEY (rental_id);

-- Sequence anlegen: Neue Verleihvorgänge erhalten Ids ab 100000
CREATE SEQUENCE evaluation_rental_seq START WITH 100000;
ALTER TABLE evaluation_rental
    MODIFY rental_id DEFAULT evaluation_rental_seq.NEXTVAL;

-- Neue Spalte Penalty hinzufügen
ALTER TABLE evaluation_rental ADD penalty NUMBER(5,2);

-- Check Constraints in evaluation_rental hinterlegen
ALTER TABLE evaluation_rental ADD CONSTRAINT eval_rental_dates_ck
    CHECK (return_date IS NULL OR return_date >= rental_date);
ALTER TABLE evaluation_rental ADD CONSTRAINT eval_rental_penalty_ck
    CHECK (penalty >= 0);
```

UE06_01_04.sql

2. PL/SQL Prozeduren **(10,5 Punkte** – 0,5 + 3 + 2 + 3 + 2**)**

Im Pilotbetrieb (siehe 1.4) sollen Verleih und Rückgabe über gespeicherte Prozeduren abgewickelt werden. Jeder neue Verleihvorgang wird dabei der Mitarbeiterin bzw. dem Mitarbeiter zugeordnet, die bzw. der ihn durchführt. Im Pilotbetrieb ist das der angemeldete Datenbankbenutzer, den die **Systemfunktion** `USER` liefert.

sich selbst inserten

1. Legen Sie in der Tabelle `staff` einen neuen Eintrag für sich selbst an. Verwenden Sie als `username` Ihren Datenbankbenutzer (`USER`) und ordnen Sie sich der Filiale mit der ID 1 zu. Alle anderen Spalten können Sie beliebig befüllen. Schreiben Sie die Daten fest.

2. Erstellen Sie danach die Prozedur `rent_film`, die einen Film verleiht (in `evaulation_rental`), mit folgender Spezifikation und Logik:

```
CREATE OR REPLACE PROCEDURE rent_film (
    p_customer_id  IN evaluation_rental.customer_id%TYPE,
    p_film_id      IN film.film_id%TYPE)
IS . . .
```

   Die Prozedur holt zuerst aus der Tabelle `staff` die `staff_id` und die Filiale `store_id` der angemeldeten Person (`USER`). Anschließend ermittelt sie für diese Filiale das „nächstbeste“ freie Exemplar (mit der kleinsten `inventory_id`) des gewünschten Films (`p_film_id`). Frei ist ein Exemplar, wenn es in `evaluation_rental` keinen Verleihvorgang ohne Rückgabedatum gibt. Ist kein Exemplar frei, gibt die Prozedur eine Meldung aus (`DBMS_OUTPUT`) und speichert nichts. Sonst legt sie **einen neuen Verleihvorgang** mit dem aktuellen Datum als Verleihdatum für den angegebenen Kunden `p_customer_id` (es muss nicht überprüft werden, ob diese/r existiert) in `evaluation_rental` an. Geben Sie die neue `rental_id` sowie das verliehene Exemplar (`inventory_id`) aus (`DBMS_OUTPUT`).

   Hinweis: Die von der Datenbank vergebene `rental_id` (siehe Sequence in 1.4) erhalten Sie mit `INSERT ... RETURNING rental_id INTO variable`.

3. Testen Sie Ihre Prozedur `rent_film` aus 2.2 für folgende Fälle:

   a) Verleihen Sie einen verfügbaren Film so oft, bis in Ihrer Filiale (ID 1) kein Exemplar mehr frei ist.
   b) Versuchen Sie einen Film zu verleihen, von dem es in Ihrer Filiale (ID 1) kein Exemplar gibt.
   c) Überlegen Sie außerdem, was passieren würde, wenn ein Datenbankbenutzer ohne Eintrag in `staff` die Prozedur aufruft.

   Dokumentieren und erklären Sie das Verhalten in allen Fällen.

4. Erstellen Sie die Prozedur `return_film` für die Rückgabe eines Films (in `evaulation_rental`), mit folgender Spezifikation und Logik:

```
CREATE OR REPLACE PROCEDURE return_film (
    p_rental_id    IN evaluation_rental.rental_id%TYPE,
    p_return_date  IN DATE DEFAULT SYSDATE)
IS . . .
```

   Wird beim Aufruf kein Rückgabedatum übergeben, gilt das aktuelle Datum, andernfalls das übergebene Datum (`p_return_date`). Für die übergebene `p_rental_id` soll das Rückgabedatum und eine etwaige Penalty (Spalte `penalty`) gespeichert werden. Geben Sie zudem die berechnete Penalty und etwaige Verspätungstage aus (`DBMS_OUTPUT`).
   Angaben zur Berechnung: Die Verspätung in Tagen ergibt sich aus den angebrochenen Leihtagen abzüglich der erlaubten Leihdauer des Films (so wie in 1.1). Die Penalty beträgt die Verspätung in Tagen multipliziert mit der Leihgebühr des Films (`rental_rate`), höchstens jedoch die Wiederbeschaffungskosten (`replacement_cost`); ohne Verspätung beträgt sie 0.
   Hinweis: Auf eine nicht vorhandene Verleihung oder eine bereits zurückgegebene Verleihung muss nicht geprüft werden.

5. Testen Sie die Prozedur `return_film` aus 2.4, indem Sie für Kundin mit ID 1 vier Verleihungen durchführen (durch Aufruf der Prozedur `rent_film` aus 2.2; notieren Sie sich die jeweilige `rental_id`) und die Vorgänge auf unterschiedliche Weise zurückgeben:
   a) ohne Angabe eines Rückgabedatums (= aktuelles Datum)
   b) nach zehn Tagen (`SYSDATE + 10`)
   c) nach sechzig Tagen
   d) mit dem gestrigen Datum (`SYSDATE - 1`)

   Dokumentieren und erklären Sie das Verhalten in allen Fällen.

3. Regelmäßige Aufgaben **(6 Punkte** – 0,5 + 1 + 0,5 + 3 + 1**)**

Sie erhalten die Aufgabe, die Aktivität in der Datenbank zu analysieren. Als ersten Anlaufpunkt möchten Sie die Anzahl der offenen Sessions überwachen.

1. Erstellen Sie eine Abfrage, welche die Anzahl der momentan offenen Benutzer-Sessions ermittelt. Analysieren Sie dazu die Tabelle `v$session`.

2. Erstellen Sie eine Tabelle `monitor_sessions` für die Protokollierung der Abfrage über die Anzahl der Sessions. Fügen Sie zur Anzahl auch noch einen Zeitstempel hinzu und sehen Sie auch eine Spalte für das Einfügen eines Primärschlüssels vor. Legen Sie dafür eine eigene Sequenz an und verwenden Sie diese.

3. Erstellen Sie ein Insert-Statement, das Sie aufrufen können und die Anzahl der offenen Benutzer-Sessions mit einem Zeitstempel in Ihre Logging-Tabelle einfügt.

4. Erstellen Sie einen Job mit dem Oracle-Scheduler, der Ihr Statement aus Beispiel 3.3 im Rahmen eines anonymen PL/SQL-Blocks alle 30 Minuten ausführt und so die Anzahl der offenen Sessions protokolliert. Stellen Sie gleichzeitig sicher, dass der Job nach sieben Tagen nicht weiter ausgeführt wird (`end_date`). **Legen Sie den Job rechtzeitig vor der Abgabe an,** damit für Bespiel 3.5 genügend Messwerte entstehen. Weisen Sie mit Abfragen auf `user_scheduler_jobs` und `user_scheduler_job_run_details` nach, dass der Job aktiviert ist und erfolgreich gelaufen ist, und geben Sie die Ergebnisse ab. Läuft der Job nicht erfolgreich, dokumentieren Sie den Fehler aus den Run-Details und erklären Sie die Ursache.
   Hinweis: Aufruf mittels benannter Prozedur ist aufgrund eingeschränkter Rechte nicht möglich.

5. Erstellen Sie auf Basis der Daten, die Ihr Job aus Bsp. 3.4 in der Tabelle `monitor_sessions` protokolliert hat, eine analytische Abfrage, die einen gleitenden Durchschnitt (innerhalb 3 Stunden zuvor und 3 Stunden danach) ermittelt. Geben Sie die Abfrage und das Ergebnis mit mindestens 12 Messungen ab.
   Hinweis: Dieses Beispiel baut auf Beispiel 3.4 auf, manuell eingefügte Datensätze sind nicht zulässig.